

Pertemuan 2

Fungsi dan Struktur Kontrol

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Hari ini

- Fungsi sebagai subprogram.
- def, parameter, return, dan scope.
- Memecah masalah menjadi bagian kecil.

Dari M1 ke M2

- M1: ekspresi, variabel, input/output, percabangan, dan pengulangan.
- M2: potongan logika yang sama mulai diberi nama.
- Tujuannya: program tidak hanya berjalan, tetapi juga mudah dibaca dan digunakan ulang.

Batas materi hari ini

- Fungsi, parameter, argumen, dan return.
- Scope lokal dan global.
- Multiple return dan unpacking.
- Fungsi sebagai objek dan lambda.

Rujukan utama untuk M2

Rujukan	Yang diambil	Cara kita pakai di kelas
Miller dan Ranum	Problem solving: masalah, algoritma, program, control structures, functions.	Fungsi dipahami sebagai cara mengubah algoritma menjadi unit kerja yang jelas.
Pine Ch. 7	Functions, arguments, return values, anonymous functions, dan contoh scientific computing.	Fungsi dipakai untuk menghitung objek matematis: polinom, jarak, norma, dan ringkasan vektor.
McKinney Ch. 3	Functions, namespaces, returning multiple values, functions as objects, lambda.	Fungsi dipakai sebagai alat untuk membangun pipeline pengolahan data.

Prinsip

Materi M2 tidak mengejar banyak sintaks. Yang ditekankan: desain fungsi yang jelas, dapat dipanggil ulang, dan dapat diuji.

Dari algoritma ke subprogram

Tanpa fungsi

- Satu program panjang.
- Banyak baris berulang.
- Sulit tahu bagian mana yang salah.
- Sulit dipakai untuk data lain.

Dengan fungsi

- Satu tugas diberi nama.
- Input dan output dibuat eksplisit.
- Dapat diuji secara terpisah.
- Dapat dipakai dalam fungsi lain.

Fungsi adalah cara membuat potongan algoritma menjadi unit yang bisa dipanggil kembali.

```
def nama_fungsi(parameter_1, parameter_2):  
    langkah_1  
    langkah_2  
    return hasil
```

Bagian penting

- `def`: mendefinisikan fungsi.
- Nama fungsi: sebaiknya berupa kata kerja atau nama konsep.
- Parameter: nama variabel di dalam definisi fungsi.

Saat dipanggil

- Argumen: nilai yang dikirim saat fungsi dipanggil.
- `return`: nilai yang dikirim kembali ke pemanggil.
- Fungsi selesai ketika mencapai `return`.

Contoh 1: fungsi dari rumus matematika

$$p(x) = 2 - 3x + x^2$$

$$p(4) = 2 - 12 + 16 = 6$$

```
def polinom_sederhana(x):  
    return 2 - 3*x + x**2  
  
hasil = polinom_sederhana(4)  
print(hasil)
```

Perhatikan

Nilai x pada fungsi adalah parameter. Nilai 4 saat pemanggilan adalah argumen.

Parameter dan argumen

```
def luas_persegi_panjang(panjang, lebar):  
    return panjang * lebar  
  
A = luas_persegi_panjang(5, 3)  
B = luas_persegi_panjang(lebar=3, panjang=5)
```

Positional argument

Urutan argumen penting. Nilai pertama masuk ke parameter pertama.

Untuk fungsi matematis, nama parameter yang jelas mengurangi salah tafsir.

Keyword argument

Nama parameter disebutkan langsung. Urutan tidak lagi menjadi fokus.

Empat bentuk fungsi

Bentuk	Contoh	Kapan berguna
Tanpa argumen, tanpa return	<pre>def salam(): print(...)</pre>	Menampilkan pesan atau instruksi.
Tanpa argumen, dengan return	<pre>def pi(): return 3.14159</pre>	Menghasilkan konstanta atau konfigurasi.
Dengan argumen, tanpa return	<pre>def tampil(x): print(x)</pre>	Melaporkan hasil, bukan menghitung lanjut.
Dengan argumen, dengan return	<pre>def f(x): return x**2</pre>	Menghasilkan nilai untuk dipakai lagi.

Catatan

Pada komputasi matematika, bentuk paling sering adalah: fungsi dengan argumen dan dengan return.

print tidak sama dengan return

```
def g(x):  
    return x**2 - 1  
  
def h(x):  
    print(g(x))
```

```
y = h(3)  
print(y)  
  
z = h(3) + 2 # error
```

Ide utama

print hanya menampilkan. Jika tidak ada return, fungsi mengembalikan None. Karena itu hasilnya tidak bisa langsung dipakai pada operasi aritmatika.

Fungsi yang mengembalikan lebih dari satu nilai

```
def stat_ringkas(a, b, c, d):  
    nilai = [a, b, c, d]  
    minimum = min(nilai)  
    maksimum = max(nilai)  
    rata = sum(nilai) / len(nilai)  
    rentang = maksimum - minimum  
    return minimum, maksimum, rata, rentang  
  
mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

Apa yang terjadi?

Python sebenarnya mengembalikan satu objek bertipe tuple. Unpacking membagi isi tuple ke beberapa variabel.

Tuple dan unpacking

```
hasil = stat_ringkas(4, 7, 1, 9)
print(hasil)
print(type(hasil))
```

```
minimum, maksimum, rata, rentang = hasil

# jumlah variabel harus cocok
x, y = hasil # ValueError
```

Mengapa penting?

Dalam komputasi, satu proses sering menghasilkan beberapa informasi: parameter, galat, jumlah iterasi, atau status konvergensi.

Scope: variabel lokal dan global

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Variabel lokal

x di dalam fungsi hanya hidup di dalam fungsi tersebut.

Variabel global

x di luar fungsi tetap bernilai 10. Nilai ini tidak otomatis berubah hanya karena ada x lokal.

Mengapa variabel global perlu hati-hati?

```
total = 0

def tambah(x):
    global total
    total = total + x
    return total
```

Masalah yang sering muncul

- Hasil fungsi bergantung pada riwayat pemanggilan.
- Test menjadi tidak mandiri.
- Kesalahan sulit dilacak karena nilai dapat berubah dari tempat lain.

Saran awal

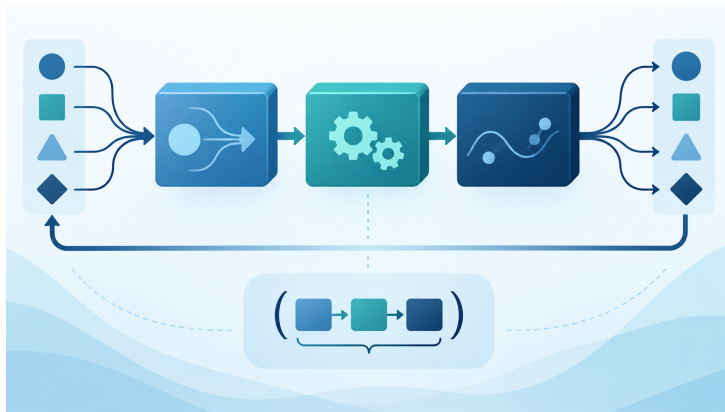
Utamakan fungsi yang menerima input secara eksplisit dan mengembalikan output secara eksplisit.

$$E(p, q) = \frac{1}{1 + d(p, q)^2}$$

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

```
def jarak(p, q):  
    dx = p[0] - q[0]  
    dy = p[1] - q[1]  
    return (dx**2 + dy**2)**0.5  
  
def energi(d):  
    return 1 / (1 + d**2)  
  
def energi_dua_titik(p, q):  
    return energi(jarak(p, q))
```

Komposisi fungsi



Jika g menerima x dan f menerima hasil dari g , maka $(f \circ g)(x) = f(g(x))$. Dalam Python, ide yang sama muncul saat satu fungsi memanggil fungsi lain.

Fungsi sebagai objek

```
def proses(x, f):  
    return f(x)  
  
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
print(proses(5, kuadrat))  
print(proses(5, tambah1))
```

Makna penting

Nama kuadrat tanpa tanda kurung adalah objek fungsi. Objek itu dapat dikirim sebagai argumen ke fungsi lain.

List fungsi dan pipeline

```
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
def pipeline(x, ops):  
    hasil = x  
    for f in ops:  
        hasil = f(hasil)  
    return hasil  
  
ops = [abs, kuadrat, tambah1]  
print(pipeline(-3, ops))
```

Baca alurnya

Nilai -3 diproses oleh `abs`, lalu `kuadrat`, lalu `tambah1`. Ini mirip alur transformasi data.

Lambda: fungsi pendek untuk operasi pendek

$$f(x) = 3x^2 - 2x + 1$$

```
f = lambda x: 3*x**2 - 2*x + 1
```

```
xnilai = range(-3, 4)
y = [f(x) for x in xnilai]

z = list(map(lambda x: 3*x**2 - 2*x + 1,
              xnilai))
```

Gunakan secara wajar

Lambda cocok untuk fungsi satu baris yang jelas. Jika logika mulai panjang, fungsi bernama lebih baik.

List comprehension untuk evaluasi fungsi

```
def f(x):  
    return 3*x**2 - 2*x + 1  
  
xnilai = [-3, -2, -1, 0, 1, 2, 3]  
y = [f(x) for x in xnilai]
```

Cara membaca

Untuk setiap x di $xnilai$, hitung $f(x)$, lalu simpan hasilnya ke list baru.

Batas materi

Hari ini list comprehension dipakai secukupnya untuk memahami fungsi. Struktur data list dan array akan dibahas lebih kuat pada M3.

Merancang fungsi yang mudah diuji

```
def jarak_1d(x, y):  
    return abs(x - y)  
  
assert jarak_1d(3, 3) == 0  
assert jarak_1d(3, 8) == 5  
assert jarak_1d(-2, 3) == 5
```

Kriteria awal

- Input jelas.
- Output jelas.
- Tidak bergantung pada variabel global.
- Dapat diuji dengan contoh yang jawabannya diketahui.

Checklist membaca fungsi

Saat melihat fungsi baru, tanyakan lima hal ini

- ➊ Apa nama tugas yang dikerjakan fungsi?
- ➋ Apa input yang dibutuhkan?
- ➌ Apa output yang dikembalikan?
- ➍ Apakah fungsi hanya mencetak, atau benar-benar mengembalikan nilai?
- ➎ Apakah fungsi bergantung pada keadaan di luar dirinya?

Untuk mahasiswa Matematika

Biasakan menulis fungsi sebagai padanan komputasional dari objek matematika: fungsi, transformasi, norma, jarak, ringkasan, atau operator.

Batas antarpertemuan

Pertemuan	Fokus	Tidak dipaksakan di M2
M1	Tipe data, variabel, operasi, if, for, while.	Fungsi Python formal belum menjadi fokus.
M2	Fungsi, parameter, return, scope, multiple return, fungsi sebagai objek, lambda.	Dictionary, NumPy array, debugging sistematis, exception, rekursi.
M3	List, dictionary, pseudocode, array NumPy, comprehension.	Big-O dan sorting belum menjadi pusat.
M4	Debugging, testing, exception, halt, rekursi, Newton.	Materi ini tidak diulang sebagai inti M2.

Contoh latihan terarah: ringkasan empat bilangan

```
def stat_ringkas(a, b, c, d):  
    nilai = [a, b, c, d]  
    minimum = min(nilai)  
    maksimum = max(nilai)  
    rata = sum(nilai) / len(nilai)  
    rentang = maksimum - minimum  
    return minimum, maksimum, rata, rentang
```

Yang harus dijelaskan mahasiswa

- Mengapa fungsi ini memiliki empat parameter?
- Mengapa output-nya dikembalikan, bukan dicetak?
- Bagaimana unpacking dilakukan saat fungsi dipanggil?

Contoh latihan terarah: fungsi vektor sederhana

```
def norm1(v):  
    total = 0  
    for x in v:  
        total = total + abs(x)  
    return total  
  
print(norm1([3, -4, 1, 2]))
```

Catatan

Contoh ini memakai list secukupnya sebagai wadah data. Struktur array dan operasi matriks dibahas lebih formal setelah M2.

Materi quiz

- Membaca definisi fungsi dan memprediksi output.
- Membedakan parameter dan argumen.
- Membedakan `print`, `return`, dan `None`.
- Menelusuri scope lokal dan global.
- Menentukan hasil unpacking dan pemanggilan fungsi sebagai argumen.

Cara belajar

Kerjakan prediksi dahulu di kertas atau catatan. Setelah itu jalankan Python untuk memeriksa dan menuliskan koreksi pemahaman.

Yang dikumpulkan

1. Buat empat bentuk fungsi sesuai materi hari ini.
2. Buat fungsi `polinom(x, koef)` untuk mengevaluasi polinom.
3. Buat fungsi `stat_ringkas` yang mengembalikan beberapa nilai dan gunakan unpacking.
4. Buat contoh scope lokal dan global, lalu jelaskan hasilnya.
5. Buat `proses(x, f)` dan `pipeline(x, ops)` untuk menerapkan fungsi sebagai objek.

Format

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang dan diberi komentar singkat.

Hal yang perlu dikuasai

- Membuat fungsi dengan input dan output jelas.
- Menggunakan `return` secara tepat.
- Menelusuri scope.
- Mengirim fungsi sebagai argumen.

Arah M3

- Representasi algoritma.
- List, tuple, dictionary.
- Pseudocode.
- Array NumPy dan comprehension.

Fungsi yang baik membuat program lebih dekat dengan cara berpikir matematis: jelas inputnya, jelas transformasinya, jelas outputnya.